

]
Development[Development]
Validation[Get validation from Requester]
Implementation[Implementation]
Completed[Completed]
FeatureTriage[Triage]
FeaturePlanning[Planning]
FeatureScheduling[Scheduling]
FeatureQueued[Queued]
AdministrativeTriage[Triage]
AdministrativeDevelopment[Development]

Triage

- Stage DRI: Dylan
- Purpose:
 - to determine non-technical validity/feasibility of request
 - to determine if more information is needed before it can move to next stage
 - to determine if needed approvals are present
 - to add labels indicating customer type, system impacted, priority, if it is a roadmap item
- Required artifacts:
 - Labels
 - Approval (if required)

Here the DRI will gather the needed information from the request (if required) and determine if we can move forward on the issue. The DRI should also determine if all needed approvals are present. The general logic used is:

- If the request aligns with something on the support roadmap and seems congruent with the intention, then no approval required
- If the request aligns with something on the support roadmap but is not congruent with the intention, then the DRI needs to ask for one of the following:
 - The support roadmap be updated
 - Support leadership approval be documented on the request
- If the request does not align with something on the support roadmap, then the DRI needs to ask for an appropriate level of approval from support leadership based on:
 - the amount of work
 - any danger to planned work
 - effect on customer or support workflows

If any labels are missing from the issue, the DRI should add them at this time.

If we cannot proceed, the DRI will state why on the issue via a comment and close the issue.

If every is good to go, the DRI will move the stage to Planning.

Planning

{{% alert title="Note" color="primary" %}}

This stage is skipped if it is a Administrative type issue

{{% /alert %}}

- Stage DRI: Jason
- Purpose:
 - to determine technical validity/feasibility of request
 - to determine if more information is needed before it can move to next stage
 - to generate a gameplan
- Required artifacts:
 - Gameplan
 - RICE score

Here, the DRI will write up a gameplan for the issue (and post it on the issue as a comment). If any labels are missing from the issue, the DRI should add them at this time.

A gameplan should include the following information:

- RICE score
- notes giving the Development DRI of the request enough information to get started
- rough breakdown of timelines
- rough breakdown of steps needed to complete
- any other useful information

If we cannot proceed, the DRI will work with the requester to remedy the blockers from proceeding.

If everything is good to go, the DRI will move the stage to Scheduling.

Scheduling

{{% alert title="Note" color="primary" %}}

This stage is skipped if it is a Bug or Administrative type issue

{{% /alert %}}

- Stage DRI: Lyle, Dylan, and Jason
- Purpose:
 - to determine bandwidth validity/feasibility of request
 - to determine iteration(s) and milestone(s) it will be worked in
 - to determine who will work the request in subsequent stages
- Required artifacts:
 - Iteration and milestone this will start work in
 - Who will work the request in subsequent stages

Here the DRIs will discuss development timelines for the changes. This is done on a weekly cadence.

Once determined, an iteration and milestone will be added to the issue stating the deployment

period.

If the iteration set is the current iteration, then the DRIs will move the request to the Development stage (assigning it to the Development DRI). Otherwise, the request will be moved to the Queued stage (to wait for its iteration to occur).

Queued

- Stage DRI: Lyle, Dylan, and Jason
- Purpose:
 - Indicate the request is ready to be worked but is waiting on its iteration to begin
- Required artifacts: None

Issues will sit here until their iteration has begun. Once the iteration has begun, the DRIs should assign the request to the Development DRI for the issue and change its stage to Development.

Development

- Stage DRI: Assigned during Scheduling or Queued stage
- Purpose:
 - to implement into staging/sandbox and perform testing
 - to ready staging/sandbox environments for validation
 - to provide notes on what was done
 - to get validation from the requester
- Required artifacts:
 - information on changes made in staging/sandbox environments
 - validation

It is here you will work on implementing the changes into the correct place for testing. This is normally the sandbox. As you perform changes, please indicate what was changed (and where) as comments on the issue (ideally in one thread).

Once you have gotten everything setup and done your testing, you will need to have it all validated by the requester of the issue. See Validation for more information.

Validation

```
{{% alert title="Note" color="primary" %}}
```

Validation is not done for Administrative type issues. Instead, use the label Validation::Skipped to indicate validation was not needed.

```
{{% /alert %}}
```

Here, the requester of the issue validates the changes we have made will resolve their request. This is traditionally done in a sandbox environment.

To begin this, add the label Validation::Requested on the issue. From there, you need to ping the requester of the issue to ask them to validate the changes in the sandbox/testing

environment. Make sure to include any information they might need, such as a link to the credentials to use or specific steps they need to use in their validation process.

If the issue requester states it has problems, add the label Validation::Rejected to the issue. You will then work on rectifying the problems. Once you are ready to re-validate, restart the process from the beginning.

If the issue requester validates the changes, add the label Validation::Received to the issue, then change the stage to Implementation.

Implementation

- Stage DRI: Assigned during Scheduling or Queued stage
- Purpose:
 - generate a technical blueprint
 - implement changes into production environments
 - confirming on what goes live at what dates (if applicable)
- Required artifacts:
 - Technical blueprint
 - Implementation of changes into production environments

Here you will generate a technical blueprint and implement the changes into production.

A technical blueprint should go over, in detail, everything that was changed. Anyone looking at the blueprint should be able to perfectly reproduce what you did. This means linking to all created MRs, detailing any non-MR changes made, etc.

The actual implementation is usually done via MRs, but can vary from change to change.

Once all implementation tasks are completed (for items using deployments, merging the MR is sufficient), change the stage to Completed.

Completed

- Stage DRI: Assigned during Scheduling or Queued stage
- Purpose: State everything is done
- Required artifacts: N/A

This indicates the work is done. An issue is moved to this stage by us when all changes are made in production (or queued up for the next deployment).

The only work left to be done in this stage is to close the issue

Blocked

- Stage DRI: The person who moved it to this stage
- Purpose:
 - to indicate an issue is blocked in some way
 - to indicate what is blocking the issue
 - to indicate the stage where it became blocked

- Required artifacts:
 - information indicating at what stage it was previously before being blocked
 - information indicating what is blocking it
 - weekly checkins on the status of the block

This is a special stage that is used when something has blocked all movement on an issue. This could be related to missing approvals, waiting on a tool to be procured, etc.

In this stage, your task is to check on what is blocking it to determine if it is ready to be unblocked.

If it is ready to be unblocked, move it back to the last stage it was in prior to being moved to the Blocked Stage.

Backlogged

- Stage DRI: The person who moved it to this stage
- Purpose:
 - to indicate an issue is backlogged
- Required artifacts:
 - information indicating at what stage it was previously before being backlogged

This is a special stage that is used when an issue is backlogged. This normally means that while there is a desire to work it, it is to be handled in the far future (beyond the next 10 iterations) or at an unknown future date.

If it is ready to be unbacklogged, move it back to the last stage it was in prior to being moved to the Backlogged Stage.

SECTION: security/customer-support-operations/workflows/gitlab/working-mrs.md

Working merge requests is not too different than how you would normally use a merge request in GitLab.

The important things to remember are:

- Always get a thorough peer review. This should be required on the merge request (as it needs approval from someone who has not made a commit).
- During peer review, you can make suggestions. But never commit them. Doing so makes you no longer able to approve the merge request (and thus now requires another peer to review it).

SECTION: security/customer-support-operations/workflows/incident_io/_index.md

{{% alert title="Note" color="primary" %}}

incident.io changes are classified as ad-hoc. Once changes are made in incident.io, they are

live.

This can mean changes can cause severe problems if you have not thoroughly tested them.

{{% /alert %}}

Making changes

{{% alert title="Note" color="danger" %}}

Always, always, ALWAYS perform thorough and complete testing for incident.io changes.

{{% /alert %}}

Currently, changes to incident.io are made in incident.io itself.

SECTION: security/customer-support-operations/workflows/ruby-gems/_index.md

{{% pageinfo color="info" %}}

This is generalized workflow information for the ruby gems we develop. If a specific gem has special workflows, a dedicated page will be made for it.

{{% /pageinfo %}}

{{% alert title="Note" color="danger" %}}

Gem changes are classified as ad-hoc. Once changes are made to the repo and the new version of the gem is pushed, they will be used moving forward.

This can mean changes can cause severe problems if you have not thoroughly tested them.

{{% /alert %}}

Making changes

{{% alert title="Note" color="danger" %}}

Always, always, ALWAYS perform thorough and complete testing for gem changes.

{{% /alert %}}

Making the changes would be done by modifying the files within the project. You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Once the MR is merged, you need to push the gem. This is done by navigating to the repo on your local computer, updating the repo, building the gem, and pushing the new version. It would look

something like this (keep in mind your version number will likely differ):

```
bash
cd ~/dev/gitlab-support-readiness/gitlabsupportreadinessgem
git pull
gem build ./gitlabsupportreadiness.gemspec
gem push gitlabsupportreadiness-1.0.130.gem
```

SECTION: security/customer-support-operations/workflows/salesforce/_index.md

SECTION: security/customer-support-operations/workflows/salesforce/account-merges.md

^{Introduced via gitlab-com/support/support-ops/support-ops-project1694}

Overview

This page is intended to provide guidance for when you must review a Salesforce (SFDC) customer account merge request. To ensure that Zendesk is kept up to date with the correct details about our customers and to ensure that we provide a consistent support experience even when changes are being made to customer accounts by Sales, we have created a process to review these requests. An example of such a request can be found [here](#).

Reviewing SFDC Account Merge Requests

1. Initial Review of Organizations in Zendesk

- Check Support Entitlement:
 - Review the target organization to ensure they have support entitlement. Ensure that the target organization is neither a prospect nor in a Support Hold state.
 - Note: If the target organization does not meet this criteria, we should not proceed with the account merge until the customer is eligible for support. Instead, respond back to the original account merge request issue informing Sales Ops that the target organization is not currently eligible for support and that they should resolve this before proceeding with the account merge.
- Check for any special configurations:
 - Check if the source organization has a Contact Management Project (CMP) or if it's a shared organization.
 - Action: If either condition is true, we can proceed with the account merge. However, Customer Support Operations should inform Sales Ops in the issue that the target organization will not automatically have the same configuration. The customer will need to raise a ticket with Customer Support Operations to explicitly request that the target organization is updated after the account merge has taken place.
- Once eligibility for the Account Merge has been confirmed:
 - Customer Support Operations should reply back to the original issue, providing their approval and sign off on the account merge.

- Customer Support Operations should add a note to the source organization documenting the target organization. These can be added as Customer Support Operations organization notes.

1. Post-Merge Actions

- Communication:
 - Once the account merge is complete, Sales Ops should notify Customer Support Operations about the successful merge.
- Merging Contacts and Tickets:
 - Customer Support Operations should use the Zendesk (ZD) merge org feature to consolidate contacts and tickets from the source organization into the target organization.

SECTION: security/customer-support-operations/workflows/salesforce/skus.md

{{% alert title="Note" color="danger" %}}

This directly impacts the calculated support entitlement of our customers in Zendesk. Exercise extreme caution in making changes.

{{% /alert %}}

When a SKU (product rate charge) is created, renamed, deleted, or redefined, we need to update both the gitlab-support-readiness gem and Product Rate Charge Name mapping google sheet (internal only) to reflect it.

Remember changes here can have an impact on the ZD-SFDC sync, so consider that as well when you are making changes to the SKU mapping.

SECTION: security/customer-support-operations/workflows/zendesk/_index.md

SECTION: security/customer-support-operations/workflows/zendesk/agent-sync.md

{{% alert title="Note" color="primary" %}}

Agent sync changes are classified as ad-hoc. Once changes are made to the repo, they will be applied on the next run.

{{% /alert %}}

Runtimes

Agent sync runs during the following times:

- Zendesk Global: Runs at 0000 UTC every day (0 0)

- Zendesk US Government: Runs a 0000 Pacific every day (0 0)

Request for changes

Requests to make changes to the agent sync (or components therein) require specific levels of approval:

- Changes to signatures: Support Director
- Changes to displayed name: Support Director
- Changes to mid-ticket feedback information: Support Director
- Changes to tagging: Support Director and Support Data Specialist
- Changes to Zendesk field values: Sr. Support Engineering Manager - Operations
- Anything else: Support Manager

The approval is required to proceed. If it has not been granted, the request should be rejected.

Making changes

Making the changes would be done by modifying the script files within the project. You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

SECTION: security/customer-support-operations/workflows/zendesk/api.md

{{% alert title="Note" color="primary" %}}

Anything involving the Zendesk API is classified as ad-hoc.

{{% /alert %}}

{{% alert title="Note" color="danger" %}}

All Zendesk API tokens are admin level tokens. They are extremely dangerous and should only be issued when absolutely necessary.

{{% /alert %}}

Token creation requests

{{% alert title="Note" color="primary" %}}

All token naming should be a link to the item the token was issued for.

{{% /alert %}}

All requests for an API token should be done via a access request issue.

There are two exceptions to this:

- API tokens for Support Operations team members' personal use
- API tokens for Support Operations scripts/automations/etc.

Once an access request is filed, the requester's manager must approve the request.

After that has been done, the provisioner for the instance (traditionally a Fullstack Engineer, Customer Support Operations) will review the request.

During this review, we carefully review the business reasons and use-case of each request due to the significant access level that an API token provides.

If deemed acceptable, the Fullstack Engineer, Customer Support Operations will then create the API token. The name of the token in Zendesk should be a link to the access request.

The API token will then be DM'd in Slack to the requester.

See: Zendesk API Tokens

Integration requests

```
{{% alert title="Note" color="danger" %}}
```

These are significantly harder to revoke than API tokens. We should only ever create these via the Integration bot for the instance:

- Zendesk Global

Due to the security risk, we do not currently do integration requests for Zendesk US Government.

```
{{% /alert %}}
```

All requests for an integration token should be done via a access request issue.

Once an access request is filed, the requester's manager must approve the request.

After that has been done, the provisioner for the instance (traditionally a Fullstack Engineer, Customer Support Operations) will review the request.

During this review, we carefully analyze the business reasons and use-case due to the significant access level that an API token provides. Integrations pose even higher risks and should be avoided whenever possible. While API tokens can be quickly and easily revoked, integrations cannot.

If deemed acceptable, the Fullstack Engineer, Customer Support Operations will then create the integration. The exact means for this are going to vary from integration to integration, but the key point is it will be done by logging in as the Integration bot for the Zendesk instance.

See: Zendesk OAuth Applications

SECTION: security/customer-support-operations/workflows/zendesk/apps.md

{{% alert title="Note" color="primary" %}}

All app creations are handled via an ad-hoc deployment.

All app updates are handled via a standard deployment.

{{% /alert %}}

All our apps are handled via the deployment cycle. This means updates traditionally are only applied on the first of each month.

Creating a new app

Creating a new app is a more involved process. For this, you will:

1. Setup the project (see the project setup workflow for more details)
1. Install the app in the sandbox Zendesk
1. Install the app in the production Zendesk
1. Setup the scheduled pipeline for the project

Updating an app

Updating an app is done by updating the source code of the app.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Deactivating an app

Deactivating an app requires a very manual process. When deactivating an app, you must:

1. Deactivate the app in the Zendesk instance
 1. Navigate to the Zendesk instance's admin center
 1. Navigate to Apps and integrations > Apps > Zendesk Support apps
 1. Hover over the app in question
 1. Click the gear icon
 1. Click the slider under Enabled
1. Deactivate the scheduled pipeline for the app
 1. Navigate to the app project
 1. Go to its scheduled pipelines
 1. Click the button to assume ownership of the scheduled pipeline
 1. Click the icon to edit it
 1. Uncheck the box next to Activated
 1. Click Save Changes

Packaging an app

This is done by combining all the "app code" into a zipfile. To accomplish this, you need to access the app project's source code via CLI and zip the content up. This might look something like:

```
bash
cd ~/dev/support-readiness/zendesk-global/apps/star
zip -r data/application.zip assets manifest.json translations
```

This will produce the zipfile application.zip in the data folder of your local repository (which git will ignore thanks to the .gitignore file you copied earlier).

Exception deployment

Some changes, such as bug fixes, warrant an immediate deployment.

To perform an exception deployment for an app, navigate to the app project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the app.

SECTION: security/customer-support-operations/workflows/zendesk/articles.md

We use Zendesk Guide for Knowledge base. The Knowledge base has 3 type of roles with different permissions:

Support Engineer, Knowledge Champions and Knowledge Admins.

- Support Editors: Create, update and use KB articles in tickets.
- Support Publishers: Review, publish, archive and control permissions.
- Admins: Make sure everything runs smoothly.

Creating an article

All Support Team Members can create Articles in the knowledge base in draft mode. Support Publishers and Admins can approve and publish the articles. There is no requirement to file an issue for that. For the creation of articles, please see Zendesk documentation.

Modifying an article

All Support Team Members can modify Articles in the knowledge base in draft mode. Support Publishers and Admins can approve and publish the updated articles. There is no requirement to file an issue for that. For information on modifying an article, please see Zendesk documentation.

Archiving an article

Support Publishers and Admins can archive the article directly in the Zendesk Guide. It is highly recommended that articles are archived before deletion. To see how to archive an article, please see Zendesk documentation. To view an archived article list, please see Zendesk documentation.

Restoring an article

To restore an article from archived state, Support Team member will file a Feature/Change request. Admins will follow the process to restore an article as mentioned in Zendesk Documentation.

Creating, Editing and Deleting sections

To add, edit or delete sections, Support Team members have to file a Feature/Change request. After that Admins will follow the process on request as mentioned in Zendesk Documentation.

Deleting an article

An article should be archived before deletion. To delete an article from knowledge base, Support Team members have to file a Feature/Change request after archiving it. Once the request is received, Admins can follow the process to delete an article as mentioned in [this link](#).
Note: This is a permanent action.

SECTION: security/customer-support-operations/workflows/zendesk/automations.md

{{% alert title="Note" color="primary" %}}

All automation changes are handled via a standard deployment.

{{% /alert %}}

Creating an automation

For these, you will need to create a corresponding automation file in the sync repo.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Editing an automation

For these, you will need to locate the corresponding automation file in the sync repo and make changes to it.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Deactivating an automation

To deactivate an automation, you must make the following changes in the corresponding automation file in the sync repo:

1. Move the automation from the active folder to the the corresponding location in the inactive folder (if it does not exist, create it)

1. Change the value of the active attribute in the YAML file to false

1. Change the conditions section to be:

```
yaml
- field: 'brandid'
  operator: 'isnot'
  value: 'GitLab Support'
```

1. Change the actions section to be:

```
yaml
- field: 'brandid'
  value: 'GitLab Support'
```

1. Change the value of the containsmanagedcontent attribute to false

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Deleting an automation

```
{{% alert title="Note" color="danger" %}}}
```

We do not delete automations except during an annual review of our automations. Any other time, leave it as deactivated.

If you are looking for information about the annual review of automations, please see Annual Review workflow

{{% /alert %}}

The process to delete an automation is exactly as described in the automations documentation page.

Exception deployment

To perform an exception deployment for automations, navigate to the automations project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the automations.

Repo links

- Zendesk Global sync repo
- Zendesk US Government sync repo
- Zendesk Global managed content repo
- Zendesk US Government managed content repo

SECTION: security/customer-support-operations/workflows/zendesk/ces.md

{{% alert title="Note" color="primary" %}}

All changes to the CES form are classified as ad-hoc. Once changes are merged into the default branch, the form will be generated and live.

All changes to the CES processor are classified as ad-hoc. Once the changes are merged into the default branch, the code will begin being used.

All changes to the CES Survey Zendesk components (automations, ticket fields, triggers, etc.) are handled via a standard deployment.

All changes to the CES Survey zaps are classified as ad-hoc.

{{% /alert %}}

CES Processor changes

Making the changes would be done by modifying the files within the project. You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

CES Survey Form changes

Making the changes would be done by modifying the files within the project. You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR

should enforce this).

Once the MR is merged, they will then be used to populate the form.

Zendesk Automations changes

Please see our Zendesk automations workflow

Zendesk Ticket Fields changes

Please see our Zendesk ticket fields workflow

Zendesk Triggers changes

Please see our Zendesk triggers workflow

Workato changes

Please see our Workato workflow

SECTION: security/customer-support-operations/workflows/zendesk/dev-pulse.md

{{% alert title="Note" color="primary" %}}

Dev Pulse changes are classified as ad-hoc. Once changes are merged into the default branch, they will be live.

These also usually involve changes to other components we manage.

{{% /alert %}}

Making changes

Making the changes would be done by modifying the files within the project. You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

SECTION: security/customer-support-operations/workflows/zendesk/dynamic-content.md

{{% alert title="Note" color="primary" %}}

All dynamic content changes are handled via a standard deployment. Changes here deploy to both the production and sandbox instance(s).

{{% /alert %}}


```
{{% alert title="Note" color="primary" %}}
```

This only applies to Zendesk Global, as Zendesk US Government is English only.

```
{{% /alert %}}
```

Creating dynamic content

For these, you will need to create a corresponding dynamic content file in the sync repo.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

You can use other dynamic content item files for an example of doing this, but in general the template is:

```
yaml
---
name: 'NAMEOFITEM'
placeholder: 'PLACEHOLDERZENDESKWOULDUSE'
defaultlocaleid: 1 English
variants:
- content: 'TEXTTOUSEFORLOCALE'
  localeid: 1 English
  active: true
  default: true
```

Where:

- NAMEOFITEM is the name of the dynamic content item
- PLACEHOLDERZENDESKWOULDUSE is the placeholder value Zendesk would generate. This is usually the name in all lowercase with spaces replaced with underscores and preceded by dc. (all grouped in double curly brackets). An example would be Preferred Region for Support becoming `{{dc.preferredregionforsupport}}`. If you are unsure of what your placeholder would be, a neat trick is to go into the Zendesk instance's Sandbox and create the dynamic content item in the admin panel, then look at what it generates (make sure if you do this to delete it afterwards).
- TEXTTOUSEFORLOCALE is the text the variant would use for the locale in question

Editing dynamic content

For these, you will need to locate the corresponding dynamic content file in the sync repo and make changes to it.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Deleting dynamic content

```
{{% alert title="Note" color="primary" %}}
```

Be very cautious on these. You can break a considerable amount of things by deleting dynamic content that is still actually in use.

```
{{% /alert %}}
```

To fully delete it, you must delete it from both the sync repo and the Zendesk instance(s).

For information on deleting it from Zendesk, see our dynamic content documentation.

For the sync repo, you will create a MR to delete the file (but only after deleting it from the Zendesk instance(s)). Said MR should always be peer reviewed before merging (the MR should enforce this).

Getting translations

```
{{% alert title="Note" color="primary" %}}
```

This makes the summption you're setup aligns with our recommended setup

```
{{% /alert %}}
```

Because this involves translations, which usually involve teams other than our own, we use a dynamically generated google sheet to assist with translations: Zendesk Global Dynamic Content List (internal only)

When you need translations for items, you will first run the populategsheet script (via CLI), like so:

```
bash
SPREADSHEETID='1ah9iEMwodDBrmVDEpWkigWiz5CWtDOSWwubdbt72zo' ./bin/populategsheet
```

You will then ask the translators to use the sheet to provide us the translations.

Once thhe translators are done, you can then pull that data into your local repo using the pullfromgsheet script, like so:

```
bash
SPREADSHEETID='1ah9iEMwodDBrmVDEpWkigWiz5CWtDOSWwubdbt72zo' ./bin/pullfromgsheet
```

This will edit your local repo with the changes. You can use this to then generate a MR.

Exception deployment

To perform an exception deployment for dynamic content, navigate to the dynamic content project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for dynamic content.

Repo links

- Zendesk Global sync repo

SECTION: security/customer-support-operations/workflows/zendesk/end-users.md

{{% alert title="Note" color="primary" %}}

End-user settings changes are classified as ad-hoc as the changes are made directly in Zendesk. Once changes are made in Zendesk they are live.

Changes to these can cause severe problems if you have not thoroughly tested them. Exercise extreme caution before proceeding.

{{% /alert %}}

Modifying end user settings

You will make the change directly in Zendesk. Remember any changes should also be reflected in the end-users documentaiton (as it is the single source of truth).

SECTION: security/customer-support-operations/workflows/zendesk/forms.md

{{% alert title="Note" color="primary" %}}

All form changes are handled via a standard deployment.

{{% /alert %}}

Creating a form

For these, you will need to create a corresponding form file in the sync repo.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Editing a form

For these, you will need to locate the corresponding form file in the sync repo and make changes to it.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Deactivating a form

To deactivate an form, you must make the following changes in the corresponding form file in the sync repo:

1. Move the form file from the active folder to the the corresponding location in the inactive folder (if it does not exist, create it)

1. Change the value of the active attribute in the YAML file to false

1. Change the ticketfieldids section to be:

```
yaml
- 'Status'
- 'Group'
- 'Assignee'
- 'Ticket status'
- 'Subject'
- 'Description'
```

1. Change the enduserconditions section to be []

1. Change the agentconditions section to be []

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Deleting a form

```
{{% alert title="Note" color="danger" %}}
```

We do not delete forms except during an annual review of our forms. Any other time, leave it as deactivated.

If you are looking for information about the annual review of forms, please see Annual Review workflow

```
{{% /alert %}}
```

The process to delete a form is exactly as described in the forms documentation page.

Exception deployment

To perform an exception deployment for forms, navigate to the forms project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the forms.

Repo links

- Zendesk Global sync repo
- Zendesk US Government sync repo

SECTION: security/customer-support-operations/workflows/zendesk/groups.md

{{% alert title="Note" color="primary" %}}

All group creations are handled via an ad-hoc deployment.

All group updates are handled via a standard deployment.

{{% /alert %}}

Creating a group

For these, you will need to create a corresponding group file in the sync repo.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Editing a group

{{% alert title="Note" color="danger" %}}

Due to the potential downstream impact, you should never edit a group. If you are looking to make changes to a group's name, you should make a new group and migrate users over to it.

{{% /alert %}}

Deleting a group

{{% alert title="Note" color="danger" %}}

Due to the potential downstream impact, you should never delete a group.

{{% /alert %}}

Exception deployment

To perform an exception deployment for groups, navigate to the groups project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the groups.

Repo links

- Zendesk Global sync repo

- Zendesk US Government sync repo

SECTION: security/customer-support-operations/workflows/zendesk/guide.md

{{% alert title="Note" color="primary" %}}

Guide changes are classified as ad-hoc as the changes are made directly in Zendesk. Once changes are made in Zendesk they are live.

Changes to these can cause severe problems if you have not thoroughly tested them. Exercise extreme caution before proceeding.

{{% /alert %}}

Modifying Guide settings

You will make the change directly in Zendesk. Remember any changes should also be reflected in the Guide documentaiton (as it is the single source of truth).

SECTION: security/customer-support-operations/workflows/zendesk/help-center-categories.md

{{% alert title="Note" color="primary" %}}

All Help Center category changes are handled via a standard deployment.

{{% /alert %}}

Creating a Help Center category

For these, you will need to create a corresponding Help Center category file in the sync repo.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Editing a Help Center category

For these, you will need to locate the corresponding Help Center category file in the sync repo and make changes to it.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Exception deployment

To perform an exception deployment for Help Center categories, navigate to the Help Center category project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the Help Center categories.

Repo links

- Zendesk Global sync repo
- Zendesk US Government sync repo

SECTION: security/customer-support-operations/workflows/zendesk/help-center-management-permissions-groups.md

{{% alert title="Note" color="primary" %}}

All Help Center management permissions changes are handled via a standard deployment.

{{% /alert %}}

Creating a Help Center management permission

For these, you will need to create a corresponding Help Center management permission file in the sync repo.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Editing a Help Center management permission

For these, you will need to locate the corresponding Help Center management permission file in the sync repo and make changes to it.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Exception deployment

To perform an exception deployment for Help Center management permissions, navigate to the Help Center management permissions project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the Help Center management permissions.

Repo links

- Zendesk Global sync repo
- Zendesk US Government sync repo

SECTION: security/customer-support-operations/workflows/zendesk/help-center-sections.md

{{% alert title="Note" color="primary" %}}

Help Center sections changes are classified as ad-hoc as the changes are made directly in Zendesk. Once changes are made in Zendesk they are live.

{{% /alert %}}

Creating a Help Center section

You will make the change directly in Zendesk. See the Help Center sections documentaiton for more information.

Editing a Help Center section

You will make the change directly in Zendesk. See the Help Center sections documentaiton for more information.

Moving a Help Center section

You will make the change directly in Zendesk. See the Help Center sections documentaiton for more information.

Reordering Help Center sections

You will make the change directly in Zendesk. See the Help Center sections documentaiton for more information.

Deleting a Help Center section

You will make the change directly in Zendesk. See the Help Center sections documentaiton for more information.

SECTION: security/customer-support-operations/workflows/zendesk/help-center-user-segments.md

{{% alert title="Note" color="primary" %}}

All Help Center user segments changes are handled via a standard deployment.

{{% /alert %}}

Creating a Help Center user segment

For these, you will need to create a corresponding Help Center user segment file in the sync repo.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Editing a Help Center user segment

For these, you will need to locate the corresponding Help Center user segment file in the sync repo and make changes to it.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Exception deployment

To perform an exception deployment for Help Center user segments, navigate to the Help Center user segments project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the Help Center management permissions.

Repo links

- Zendesk Global sync repo
- Zendesk US Government sync repo

SECTION: security/customer-support-operations/workflows/zendesk/macros.md

```
{{% alert title="Note" color="primary" %}}
```

Macro changes are classified as ad-hoc. Once changes are made to the repo, they sync to Zendesk.

```
{{% /alert %}}
```

While Zendesk offers a full way to manage macros via the UI, we turn to a more version controlled methodology. That being the case, we utilize the support super form, sync repos, and managed content repos.

Creating a macro

When it comes to creating a macro, there are two methodologies, one for "simple" macros and one for "complex" macros.

A simple macro is one that only would modify the following:

- Ticket assignment (or removal thereof)
- Adding tags to the ticket
- Adding a public or private comment to a ticket

If a macro would do anything other than those listed items, it is deemed "complex" at this time.

Simple macros

For the creation of simple macros, the requester would utilize the Support Super Form. A submission from there will provide the processor all the information it needs to fully create the macro (including a managed content file if required).

Complex macros

For the creation of complex macros, the requester would utilize the Support Super Form. This will result in an issue being created for you to work. The issue should contain all the needed information. The processor will have also created a managed content file for you if required.

Once you get the issue, you will need to navigate to the sync repo for the instance(s) the macro is being created on. Here, you will manually create the YAML file for the macro using the information provided in the issue.

An example YAML file would be:

```
yaml
---
title: General::Verify account self-managed or .com
active: true
description:
actions:
- field: commentvalue
```

```
  value: MANAGEDCONTENT
- field: status
  value: pending
restriction:
containsmanagedcontent: true
```

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read [Working with sync repo files](#) for more information.

Once the file is created and live on the master branch of the sync repo, the macro will automatically sync into the production Zendesk instance.

Editing a macro

For these, you will need to locate the corresponding macro file in the sync repo and make changes to it.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read [Working with sync repo files](#) for more information.

Once the file is created and live on the master branch of the sync repo, the macro will automatically sync into the production Zendesk instance.

Deactivating a macro

To deactivate a macro, you must make the following changes in the corresponding macro file in the sync repo:

1. Move the macro from the active folder to the the corresponding location in the inactive folder (if it does not exist, create it)
1. Change the value of the active attribute in the YAML file to false
1. Change the actions section to be:

```
yaml
- field: 'brandid'
  value: 'GitLab Support'
```

1. Change the value of the containsmanagedcontent attribute to false

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Once the changes are live on the master branch of the sync repo, the macro will automatically sync into the production Zendesk instance.

Deleting a macro

```
{{% alert title="Note" color="danger" %}}
```

We do not delete macros except during an annual review of our macros. Any other time, leave it as deactivated.

If you are looking for information about the annual review of macros, please see [Annual Review workflow](#)

```
{{% /alert %}}
```

The process to delete a macro is exactly as described in the [macros documentation page](#)

Repo links

- Zendesk Global sync repo
- Zendesk US Government sync repo
- Zendesk Global managed content repo
- Zendesk US Government managed content repo

SECTION: security/customer-support-operations/workflows/zendesk/maintenance.md

To maintain healthy environments, there are several maintenance tasks we must do. While the aim is to always automate these, some cannot be automated and must be done manually from time to time. If the task is automated, it will indicate as much

Enable upcoming version deprecation warning in the themes

To warn customers of a major version being unsupported soon, we display a warning on the theme when they enter their GitLab version.

To modify this, you would do the following:

- Update the data/theme/script.js file
- You are wanting to make three changes here:
 1. Update the version number and date in the upcomingunsupportedversionmessage function
 1. Ensure the following if block is uncommented:

```
javascript
if ($(this).val().split('.')[0] == xx) {
  $('#gitlabversioncheckerupcoming').show();
} else {
  $('#gitlabversioncheckerupcoming').hide();
}
```

- Where xx is the previously used version number
- 1. Ensure the previously used version number in the if block (from point 2) is updated
- Update the version in the data/theme/manifest.json file

This is done in a way to ensure it is enabled on the theme for at least a month prior to the GitLab major release. Refer to GitLab releases for more info on upcoming releases. So if the release date is 2025-05-15, you would want to ensure it is done in time for the 2025-04-01 deployment.

Disable upcoming version deprecation warning in the themes

As the version from the message is now unsupported, we need to remove the warning. To do this, simply modify the theme in the following way:

- Update the data/theme/script.js file
- Ensure the following if block is commented out:

```
javascript
if ($(this).val().split('.')[0] == xx) {
  $('#gitlabversioncheckerupcoming').show();
} else {
  $('#gitlabversioncheckerupcoming').hide();
}
```

- Where xx is the previously used version number
- Update the version in the data/theme/manifest.json file

This is done on the day of a GitLab major release. So if the release is 2025-05-15, these changes would go live on that date.

Update supported versions in the themes

We need to update the list of supported versions. This is done by modifying the theme in the following way:

- Update the data/theme/script.js file
 - Locate the definition of the supportedversions variable and modify the Array to to have the currently support versions.
- Update the version in the data/theme/manifest.json file

This is done on the day of a GitLab major release. So if the release is 2025-05-15, these changes would go live on that date.

Fix bad task ticket

^{Automated in Zendesk Global via Zendesk Global > Maintenance Tasks to run every hour at the 30 minute mark.}

^{Automated in Zendesk US Government via Zendesk US Government > Maintenance Tasks to run every hour at the 30 minute mark.}

This locates Task type tickets that have a Due Date set before the current time. Any tickets found in this state have their due date removed and are set to the type of Question.

Applicable instance:

- Zendesk Global
- Zendesk US Government

Fix false expired tags in orgs

^{Automated in Zendesk Global via Zendesk Global > Maintenance Tasks to run every hour at the 30 minute mark.}

This locates organization's containing the expired tag and a supportlevel value that is not expired. For these organizations, we need to remove the false expired tag. The script will do so after checking the following:

- Removing the expired tag does not remove all supportlevel tags on the organization
- Removing the expired tag would yeild an actual update

Applicable instance:

- Zendesk Global

Permanently delete deleted users

^{Automated in Zendesk Global via Zendesk Global > Maintenance Tasks to run twice daily via Pipeline Schedules at 0045 and 1245.}

^{Automated in Zendesk US Government via Zendesk US Government > Maintenance Tasks to run twice daily via Pipeline Schedules at 0045 and 1245.}

Deleting a user removes them from the Zendesk workspace, however it does not actually fully remove the user. As such, we need to routinely remove these deleted users from Zendesk.

Applicable instance:

- Zendesk Global
- Zendesk US Government

Purge Redis cache for globalorganizations

^{Automated in Zendesk Global via Zendesk Global > Maintenance Tasks to run every Saturday at 0030 UTC.}

This clears the Redis data within the key globalorganizations. This is needed from time to time to ensure the Redis cached data isn't stale.

Applicable instance:

- Zendesk Global

Purge Redis cache for usgovorganizations

^{Automated in Zendesk US Government via Zendesk US Government > Maintenance Tasks to run every Saturday at 0030 UTC.}

This clears the Redis data within the key usgovorganizations. This is needed from time to time to ensure the Redis cached data isn't stale.

Applicable instance:

- Zendesk US Government

Purge Redis cache for usgovusers

^{Automated in Zendesk US Government via Zendesk US Government > Maintenance Tasks to run every Saturday at 0030 UTC.}

This clears the Redis data within the key usgovusers. This is needed from from time to time to ensure the Redis cached data isn't stale.

Applicable instance:

- Zendesk US Government

SECTION: security/customer-support-operations/workflows/zendesk/org-fields.md

{{% alert title="Note" color="primary" %}}

All organization field changes are handled via a standard deployment.

{{% /alert %}}

Creating an organization field

For these, you will need to create a corresponding organization field file in the sync repo.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Editing an organization field

For these, you will need to locate the corresponding organization field file in the sync repo and make changes to it.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Deactivating an organization field

```
{{% alert title="Note" color="primary" %}}
```

Be very cautious on these. You can break a considerable amount of things by deactivating an organization field that is still actually in use.

```
{{% /alert %}}
```

To deactivate an organization field, you must make the following changes in the corresponding organization field file in the sync repo:

1. Move the organization field file from the active folder to the the corresponding location in the inactive folder (if it does not exist, create it)

1. Change the value of the active attribute in the YAML file to false

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Deleting an organization field

```
{{% alert title="Note" color="danger" %}}
```

We do not delete organization fields except during an annual review of our organization fields. Any other time, leave it as deactivated.

If you are looking for information about the annual review of organization fields, please see [Annual Review workflow](#)

```
{{% /alert %}}
```

The process to delete an organization field is exactly as described in the organization fields documentation page.

Exception deployment

To perform an exception deployment for organization fields, navigate to the organization fieldsa project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the organization fields.

Repo links

- Zendesk Global sync repo
- Zendesk US Government sync repo

SECTION: security/customer-support-operations/workflows/zendesk/provisioning.md

The method used largely depends on the type of provisioning being done.

Role based entitlement

2 days after someone starts working at GitLab, an access-request issue is generated based off their role based entitlements. We would manually provision users based off the request itself.

To do this, you will need to:

1. Create the user in Zendesk (the role to be used should be included in the role based entitlement access request issue). Ensure their groups and other such settings are accurate (see the access request issue). See Manually creating an agent for more info on creating the user.
1. Associate the correct app in Okta (see Assigning an app via Okta for more info) if required.

After you have done so, mark off the items in the access request issue.

Special request

Any special request issues to provision on either Zendesk instance not related to role based entitlements must be done via an access request issue. Do note this will require approval from the system owners (Lyle and Jason) to proceed.

See Role based entitlements for information on what to do when it comes time to provision it.

Zendesk Global light agents

This is mostly handled automatically via the Support Super Form and the Support Super Form processor.

It will handle all the Zendesk side changes. As such, in the issue created, you will just need to assign the app in Okta

Zendesk US Government light agents

As these require the tech stack provisioner to manually provision these, an Access Request issue is required.

Once one has been received and approved, the process will go as follows:

1. Submit a HelpLab request by selecting Background Checks under the People Team dropdown. On the next page, select Identity Verification or Other from the dropdown What type of support do you need? and use the prompt below to fill out your request:

- > Greetings all!
 - >
 - > Can you verify if NAME is a US Citizen? They are requesting access to the Federal Zendesk instance via ISSUE which does require it.
 - >
 - > Thanks!
 - Replace NAME with the requester's name
 - Replace ISSUE with the link to the Access Request issue
 - This should be a quick link to that form
1. Note the Access Request issue that you have contacted the People team to verify US citizenship.
 1. If the People team verifies the citizenship:
 - Create the Light Agent manually in the Zendesk US Federal instance.
 - Go into Okta and assign the app to the requester
 - Update the issue letting them know it has been provisioned.
 1. If the People team cannot verify the citizenship:
 - Comment on the Access Request issue noting citizenship could not be confirmed and that the issue will be closed, as no further action can be taken without verification from the People team.
 1. You may then provision the user in Zendesk US Federal. Do this by:
 1. Create the user in Zendesk (the role to be used should be included in the role based entitlement access request issue). Ensure their groups and other such settings are accurate (see the access request issue).
 1. Associate Zendesk Global app in Okta (see Assigning an app via Okta for more info) if required.

Zendesk Global Partner Support team

{{% alert title="Note" color="primary" %}}

Requesters must first have a fully created light agent account. IF they do not have one, direct them to have one created first.

{{% /alert %}}

This is a specialized group of light agents who are allowed to file some types of Internal Requests. These agents will need the tag partnersupportagent added on their Zendesk account.

You should only ever be asked to either add or remove that tag (and it must be via an access request issue).

Zendesk Global Order Management team

{{% alert title="Note" color="primary" %}}

Requesters must first have a fully created light agent account. IF they do not have one, direct them to have one created first.

{{% /alert %}}

This is a specialized group of light agents who are allowed to file some types of Internal Requests. These agents will need the tag ordermanagementteam added on their Zendesk account.

You should only ever be asked to either add or remove that tag (and it must be via an access request issue).

Zendesk Global OEM Management team

{{% alert title="Note" color="primary" %}}

Requesters must first have a fully created light agent account. IF they do not have one, direct them to have one created first.

{{% /alert %}}

This is a specialized group of light agents who are allowed to file some types of Internal Requests. These agents will need the tag oemmanagementteam added on their Zendesk account.

You should only ever be asked to either add or remove that tag (and it must be via an access request issue).

Deprovisioning

You will, from time to time, get a request to deprovision an agent (these will mostly stem from Offboarding tasks). To deprovision an agent, go to that agent's page in Zendesk and do the following:

- Unassign any active tickets (less than Closed) from that agent (assign them to their manager)
- Remove any of the agent's tags from Tags section
- Clear out any user fields on the profile
- Demote the agent's role to that of end-user
- Suspend the end-user
- Delete any Support Team files attached to the user (if applicable)
- After doing so, do the following on the issue requesting the deprovisioning
 - Check the corresponding boxes on the request issue

Assigning an app via Okta

To associate an app via Okta, you will add the person's email to the corresponding google group as a Member:

- Zendesk Global
- Zendesk US Government

SECTION: security/customer-support-operations/workflows/zendesk/roles.md

{{% alert title="Note" color="primary" %}}

Guide changes are classified as ad-hoc as the changes are made directly in Zendesk. Once changes are made in Zendesk they are live.

Changes to these can cause severe problems if you have not thoroughly tested them. Exercise extreme caution before proceeding.

{{% /alert %}}

Creating a role

{{% alert title="Note" color="danger" %}}

Exercise extreme caution when dealing with roles. You can cause severe security incidents or severely break agent workflows.

{{% /alert %}}

You will make the change directly in Zendesk. Remember any changes should also be reflected in the Roles documentaiton (as it is the single source of truth).

Editing a role

{{% alert title="Note" color="danger" %}}

Due to the potential downstream impact, you should never edit a role. If you are looking to make changes to a role, you should make a new role and migrate users over to it.

{{% /alert %}}

Deleting a role

{{% alert title="Note" color="danger" %}}

Due to the potential downstream impact, you should never delete a role.

{{% /alert %}}

SECTION: security/customer-support-operations/workflows/zendesk/round-robin.md

{{% alert title="Note" color="primary" %}}

Round robin changes are classified as ad-hoc. Once changes are merged into the default branch, they will be live.

These also usually involve changes to other components we manage.

This can mean changes can cause severe problems if you have not thoroughly tested them.

{{% /alert %}}

Making changes

Making the changes would be done by modifying the files within the project. You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

SECTION: security/customer-support-operations/workflows/zendesk/schedules.md

{{% alert title="Note" color="primary" %}}

Schedule changes are classified as ad-hoc as the changes are made directly in Zendesk. Once changes are made in Zendesk they are live.

{{% /alert %}}

{{% alert title="Note" color="danger" %}}

Exercise extreme caution in making schedule changes. This can directly impact contractual obligations GitLab has to its customers and agent workflows.

{{% /alert %}}

Creating a schedule

You will make the change directly in Zendesk. See the webhook documentaiton for more information.

Editing a schedule

You will make the change directly in Zendesk. See the schedule documentaiton for more information.

Deleting a schedule

You will make the change directly in Zendesk. See the schedule documentaiton for more information.

SECTION: security/customer-support-operations/workflows/zendesk/shared-organizations.md

By default, organizations are setup so that the users within it can only see and comment on their own tickets. This security measure often doesn't work for some organizations though.

Because of that, we have the ability to setup Shared Organizations, a term meaning the users in an organization have heightened permissions and can do see and/or comment on tickets that are

not theirs.

Pre-checks

```
{{% alert title="Note" color="danger" %}}
```

There is a security and legal component to this feature. As such, you must ensure all pre-checks are done every single time. When in doubt, get a Fullstack Engineer to review the situation.

```
{{% /alert %}}
```

Before we can proceed to enable a shared organization, three criteria must be met:

- The requester has confirmed which type of shared organization is desired
- The requester, on behalf of their organization, has approved the inherent security risk shared organizations present
- There is not an organization note/detail stating we will not enable a shared organization for them

If any of those criteria have not been met, we cannot proceed.

If all of those criteria are met, make sure to update the ticket metadata to indicate all of that.

Enabling a shared organization

To enable a shared organization:

1. Navigate to the organization
1. Click the text next to Users at the top-left
1. Select Can view all org tickets
1. On the drop-down below, select the correct option depending on the type they requested
1. Add a new item in the Details field containing the following:
 - > Shared organization (TYPE) enabled DATE as per TICKET
 - Replace TYPE with either read only or read+write
 - Replace DATE with today's ISO formatted date (YYYY-MM-DD)
 - Replace TICKET with the ticket's URL
1. Add a public comment on the ticket confirming it has been enabled
1. Mark the ticket as Solved

Disabling a shared organization

To disable a shared organization:

1. Navigate to the organization
1. Click the text next to Users at the top-left
1. Select Can view own tickets only
1. Add a new item in the Details field containing the following:
 - > Shared organization disabled DATE as per TICKET

- Replace DATE with today's ISO formatted date (YYYY-MM-DD)
 - Replace TICKET with the ticket's URL
1. Add a public comment on the ticket confirming it has been disabled
 1. Mark the ticket as Solved

SECTION: security/customer-support-operations/workflows/zendesk/slas.md

{{% alert title="Note" color="primary" %}}

All SLA changes are handled via a standard deployment.

{{% /alert %}}

{{% alert title="Note" color="danger" %}}

Exercise extreme caution in making SLA changes. This can directly impact contractual obligations GitLab has to its customers.

{{% /alert %}}

Creating a SLA

For these, you will need to create a corresponding SLA file in the sync repo.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Editing a SLA

For these, you will need to locate the corresponding SLA file in the sync repo and make changes to it.

You should also do this in a way that creates a MR. Said MR should